

MODULE 30

Building Basic LLM Applications using LCEL

Lectures Covered

- Lec 1 — What is Prompting and Prompt Templates
- Lec 2 — Messages and ChatPromptTemplate
- Lec 3 — Structured Output from LLM and Pydantic
- Lec 4 — Creating Schema Response using Pydantic
- Lec 5 — Output Parsers in LangChain and StrOutputParser
 - Lec 6 — JSON and Pydantic Output Parser
 - Lec 7 — Chains in LangChain
 - Lec 8 — Parallel and Conditional Chains
 - Lec 9 — What are Runnables and LCEL
 - Lec 10 — Working with Groq API
- Lec 11 — Creating APIs for App using FastAPI
- Lec 12 — Serving Groq App using FastAPI

SECTION 1: Prompting and Messages (Lectures 1–2)

Lec 1: What is Prompting and Prompt Templates

Prompting is the art of crafting inputs to an LLM to reliably get the output you want. LangChain PromptTemplates turn prompts into reusable, parameterised functions.

PromptTemplate — Simple String Prompts

```
from langchain_core.prompts import PromptTemplate

# Basic template with {variables}
pt = PromptTemplate.from_template(
    'You are a {role}. Answer the question: {question}'
)

# Format fills in variables
formatted = pt.format(role='data scientist', question='What is overfitting?')
print(formatted)
# 'You are a data scientist. Answer: What is overfitting?'

# Invoke returns a PromptValue object
prompt_val = pt.invoke({'role': 'teacher', 'question': 'Explain gradient descent'})

# Partial prompts — pre-fill some variables
partial_pt = pt.partial(role='Python expert')
result = partial_pt.invoke({'question': 'What is a decorator?'})
```

Prompt Engineering Techniques

Technique	Description Example
Zero-shot	No examples — just instruction 'Classify the sentiment: {text}'
Few-shot	2-5 examples to guide format Show input-output pairs before the query
Chain-of-thought	Ask model to reason step by step 'Think step by step: {problem}'
Role prompting	Set persona via system message 'You are an expert SQL developer'
Constraints	Specify format/length restrictions 'Answer in exactly 3 bullet points'

Lec 2: Messages and ChatPromptTemplate

Chat models use a structured message format — a list of role-content pairs. ChatPromptTemplate builds these message lists with variable placeholders.

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

# — Basic ChatPromptTemplate —————
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert in {domain}. Be concise.'),
    ('human', '{question}')
])
msgs = prompt.invoke({'domain': 'machine learning', 'question': 'What is XGBoost?'})

# — With conversation history —————
chat_prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder(variable_name='history'), # injected at runtime
    ('human', '{input}')
])

# — Few-shot chat prompt —————
few_shot = ChatPromptTemplate.from_messages([
    ('system', 'Translate English to French.'),
    ('human', 'Hello'),
    ('ai', 'Bonjour'),
    ('human', 'Good morning'),
```

```
( 'ai',      'Bonjour'),
( 'human',  '{text}'),
])
```

Message Role	Purpose Class
system	Sets the model's behaviour/persona SystemMessage
human	User's input HumanMessage
ai	Previous model response (for few-shot or history) AIMessage
placeholder	Injects a list of messages at runtime (e.g. chat history) MessagesPlaceholder

SECTION 2: Structured Output and Pydantic (Lectures 3–4)

Lec 3: Structured Output from LLM and Pydantic

LLMs return free-form text — but applications usually need structured data (JSON, typed objects). Pydantic + LangChain's `with_structured_output()` forces the model to return data matching a schema.

Why Structured Output?

- Parsing free-form text is brittle and error-prone — JSON schema is reliable.
- Applications need typed data: extract a Product with name, price, category — not a paragraph.
- `with_structured_output()` uses OpenAI function calling / JSON mode under the hood.

```
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI

# Define the expected output schema
class MovieReview(BaseModel):
    title: str = Field(description='Movie title')
    rating: float = Field(description='Rating from 1 to 10')
    summary: str = Field(description='One sentence summary')
    positive: bool = Field(description='True if overall positive')

llm = ChatOpenAI(model='gpt-4o')

# Force structured output matching the schema
structured_llm = llm.with_structured_output(MovieReview)

review = structured_llm.invoke(
    'Review: Inception is a mind-bending thriller with stunning visuals.'
)

print(type(review))          # <class '__main__.MovieReview'>
print(review.title)          # 'Inception'
print(review.rating)         # 8.5
print(review.positive)       # True
print(review.model_dump())   # dict representation
```

Lec 4: Creating Schema Responses using Pydantic

Pydantic `BaseModel` provides type validation, default values, and optional fields — making LLM output schemas robust and self-documenting.

```
from pydantic import BaseModel, Field, validator
from typing import List, Optional
from langchain_openai import ChatOpenAI

# Nested schemas
class Address(BaseModel):
    street: str
    city: str
    country: str = 'USA' # default value

class Person(BaseModel):
```

```

name:    str    = Field(description='Full name of the person')
age:     int     = Field(ge=0, le=150, description='Age in years')
skills:  List[str] = Field(description='List of skills')
address: Optional[Address] = None    # optional nested object
email:   Optional[str]    = None

# Extract structured info from unstructured text
llm = ChatOpenAI(model='gpt-4o')
extractor = llm.with_structured_output(Person)

text = '''John Smith is a 32-year-old software engineer living in
        San Francisco. He is skilled in Python, TensorFlow, and AWS.'''

person = extractor.invoke(text)
print(person.name)    # 'John Smith'
print(person.age)     # 32
print(person.skills)  # ['Python', 'TensorFlow', 'AWS']
print(person.address.city) # 'San Francisco'

# Validate output
print(person.model_dump_json(indent=2))

```

Pydantic Field best practices

Always add `description=` to every field — used by the LLM to understand what to fill.
 Use `ge=` / `le=` for numeric bounds (`ge=0` for non-negative, `le=100` for percentages).
 Use `Optional[T] = None` for fields the LLM might not be able to extract.
 Nested `BaseModel` schemas work — great for addresses, items lists, metadata.

SECTION 3: Output Parsers (Lectures 5–6)

Lec 5: Output Parsers and StrOutputParser

Output parsers sit at the end of a chain and transform the LLM's raw `AIMessage` into the data type your application needs. `StrOutputParser` is the most common — extracts the text content.

StrOutputParser — Most Common Parser

```

from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(model='gpt-4o')
prompt = ChatPromptTemplate.from_template('Tell me a fact about {topic}')
parser = StrOutputParser()

# Without parser: chain returns AIMessage object
raw_chain = prompt | llm
resp = raw_chain.invoke({'topic': 'Python'})
print(type(resp))    # <class 'langchain_core.messages.ai.AIMessage'>
print(resp.content)  # access manually

# With StrOutputParser: chain returns plain string
chain = prompt | llm | parser
text = chain.invoke({'topic': 'Python'})
print(type(text))    # <class 'str'>
print(text)          # 'Python was created by Guido van Rossum...'

# Streaming with StrOutputParser
for chunk in chain.stream({'topic': 'LangChain'}):
    print(chunk, end='', flush=True)

```

Parser	Output Type Use Case
<code>StrOutputParser</code>	<code>str</code> Any text response — most common
<code>JsonOutputParser</code>	<code>dict</code> When LLM returns JSON string
<code>PydanticOutputParser</code>	Pydantic model Typed, validated structured output

Parser	Output Type Use Case
CommaSeparatedListOutputParser	List[str] Quick list extraction
DatetimeOutputParser	datetime Extract dates from LLM output

Lec 6: JSON and Pydantic Output Parser

JsonOutputParser

```
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o')
parser = JsonOutputParser()

# Tell the model to return JSON in the prompt
prompt = ChatPromptTemplate.from_messages([
    ('system', 'Return your answer as valid JSON only. No extra text.'),
    ('human', 'Give me info about {country}: population, capital, continent.')
])

chain = prompt | llm | parser
data = chain.invoke({'country': 'Japan'})
print(type(data))          # dict
print(data['capital'])     # 'Tokyo'
print(data['population'])  # 125700000
```

PydanticOutputParser — Typed JSON

```
from langchain.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from typing import List

class Recipe(BaseModel):
    name: str = Field(description='Name of the recipe')
    ingredients: List[str] = Field(description='List of ingredients')
    prep_time: int = Field(description='Prep time in minutes')
    difficulty: str = Field(description='Easy/Medium/Hard')

parser = PydanticOutputParser(pydantic_object=Recipe)

# Parser generates format instructions automatically
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a chef. {format_instructions}'),
    ('human', 'Give me a recipe for {dish}.')
]).partial(format_instructions=parser.get_format_instructions())

chain = prompt | llm | parser
recipe = chain.invoke({'dish': 'pasta carbonara'})
print(type(recipe))          # <class '__main__.Recipe'>
print(recipe.ingredients)    # ['pasta', 'eggs', 'pancetta', ...]
print(recipe.prep_time)      # 20
```

SECTION 4: Chains in LangChain (Lectures 7–8)

Lec 7: Chains in LangChain

A Chain connects multiple components into a pipeline. LCEL (LangChain Expression Language) uses the `|` operator to compose chains declaratively — making pipelines readable, streamable, and parallelisable.

Simple Chain

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

```

llm = ChatOpenAI(model='gpt-4o')
parser = StrOutputParser()

# Translation chain
translate = ChatPromptTemplate.from_template(
    'Translate to {language}: {text}'
) | llm | parser

result = translate.invoke({'language': 'Spanish', 'text': 'Hello World'})
print(result) # 'Hola Mundo'

```

Sequential Chain — Chaining Two Prompts

```

# Chain output of one prompt to input of the next
outline_prompt = ChatPromptTemplate.from_template(
    'Create a 3-point outline for a blog post about {topic}.'
)
expand_prompt = ChatPromptTemplate.from_template(
    'Write a full blog post based on this outline:\n{outline}'
)

# Step 1: topic -> outline
outline_chain = outline_prompt | llm | parser

# Step 2: outline -> full post
# RunnablePassthrough passes the input through unchanged
from langchain_core.runnables import RunnablePassthrough

full_chain = (
    {'outline': outline_chain, 'topic': RunnablePassthrough()}
    | expand_prompt | llm | parser
)
# Note: feed the full chain a single string (topic)
blog = full_chain.invoke('Introduction to Neural Networks')

```

Lec 8: Parallel and Conditional Chains

RunnableParallel — Run Branches Simultaneously

```

from langchain_core.runnables import RunnableParallel

# Run two chains in parallel on the same input
summary_chain = ChatPromptTemplate.from_template('Summarise: {text}') | llm | parser
sentiment_chain = ChatPromptTemplate.from_template('Sentiment of: {text}') | llm | parser

parallel = RunnableParallel(
    summary = summary_chain,
    sentiment = sentiment_chain
)

results = parallel.invoke({'text': 'LangChain makes LLM apps easy to build.'})
print(results['summary']) # text summary
print(results['sentiment']) # 'Positive'

# Shorthand dict syntax (equivalent)
parallel2 = {'summary': summary_chain, 'sentiment': sentiment_chain}

```

RunnableBranch — Conditional Logic

```

from langchain_core.runnables import RunnableBranch, RunnableLambda

# Route to different prompts based on input category
tech_chain = ChatPromptTemplate.from_template('Answer technically: {question}') | llm | parser
simple_chain = ChatPromptTemplate.from_template('Answer simply: {question}') | llm | parser

branch = RunnableBranch(
    (lambda x: 'code' in x['question'].lower(), tech_chain),
    (lambda x: 'explain' in x['question'].lower(), simple_chain),
    simple_chain # default fallback
)

```

```

)

r1 = branch.invoke({'question': 'Write code to sort a list'})
r2 = branch.invoke({'question': 'Explain neural networks'})

# RunnableLambda - wrap any Python function as a Runnable
def double(x): return x * 2
double_runnable = RunnableLambda(double)
result = double_runnable.invoke(5) # 10

```

Parallel vs Sequential

RunnableParallel: multiple chains get the SAME input, run concurrently, results merged.

Sequential (|): each step gets the OUTPUT of the previous step as its input.

Use parallel when you want multiple analyses of the same text simultaneously.

Use sequential when the output of step N is the input to step N+1.

SECTION 5: Runnables and LCEL (Lecture 9)

Lec 9: What are Runnables and LCEL

Every component in LangChain implements the Runnable interface. LCEL (LangChain Expression Language) is a declarative way to compose Runnables using | — the result is itself a Runnable with all capabilities automatically.

The Runnable Interface

Method	Description Returns
.invoke(input)	Run synchronously on single input Single output
.stream(input)	Stream output tokens as they arrive Iterator of chunks
.batch(inputs)	Run on list of inputs in parallel List of outputs
.ainvoke()	Async version of invoke Awaitable output
.astream()	Async streaming Async iterator
.with_config()	Add metadata, tags, callbacks Runnable
.with_retry()	Auto-retry on failure (n times) Runnable
.with_fallbacks()	Use backup Runnables if primary fails Runnable

Key LCEL Runnables

```

from langchain_core.runnables import (
    RunnablePassthrough, # passes input through unchanged
    RunnableLambda,      # wraps any Python function
    RunnableParallel,     # runs multiple chains in parallel
    RunnableBranch,       # conditional routing
)

# RunnablePassthrough - common in RAG
chain = {'context': retriever, 'question': RunnablePassthrough()} | prompt | llm
# question flows through unchanged; context is retrieved

# RunnableLambda - custom transformations
upper = RunnableLambda(lambda x: x.upper())
chain2 = prompt | llm | StrOutputParser() | upper

# .with_retry() - resilient chains
resilient_llm = llm.with_retry(stop_after_attempt=3)

# .with_fallbacks() - use backup if primary fails
fast_llm = ChatOpenAI(model='gpt-4o-mini')
strong_llm = ChatOpenAI(model='gpt-4o')
safe_chain = strong_llm.with_fallbacks([fast_llm])

# Inspect chain structure
chain = prompt | llm | parser

```

```
chain.get_graph().print_ascii() # visualise the pipeline
```

Why LCEL over legacy chains

LCEL chains support streaming out of the box — any | chain streams automatically.

LCEL supports async natively — no extra code needed for async endpoints.

LCEL is transparent — `get_graph()` shows the full pipeline structure.

LCEL is composable — any LCEL chain IS a Runnable, so it can be piped further.

SECTION 6: Groq API and FastAPI Deployment (Lectures 10–12)

Lec 10: Working with Groq API

Groq is an inference platform that runs open-source LLMs (LLaMA 3, Mixtral, Gemma) at exceptional speed using their custom LPU hardware — typically 5-10x faster than OpenAI at lower cost.

Groq Setup

```
pip install langchain-groq

# .env
GROQ_API_KEY=gsk_your_key_here

# Get free API key: console.groq.com
```

Using Groq Models

```
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

# Available Groq models (free tier as of 2024)
llm = ChatGroq(
    model = 'llama3-8b-8192', # LLaMA 3 8B
    temperature = 0.7,
    max_tokens = 1024
)

# Other Groq models
# 'llama3-70b-8192' -> LLaMA 3 70B (stronger)
# 'mixtral-8x7b-32768' -> Mixtral 8x7B (32K context)
# 'gemma2-9b-it' -> Google Gemma 2 9B

# Same LCEL syntax as OpenAI - fully interchangeable
prompt = ChatPromptTemplate.from_template('Explain {topic} briefly.')
chain = prompt | llm | StrOutputParser()

result = chain.invoke({'topic': 'self-attention'})
print(result)

# Groq is typically 5-10x faster than OpenAI for same model family
```

Groq Model	Params Context Speed Best For
llama3-8b-8192	8B 8K Very fast Quick Q&A, chatbots
llama3-70b-8192	70B 8K Fast Complex reasoning, coding
mixtral-8x7b-32768	46.7B MoE 32K Fast Long documents, analysis
gemma2-9b-it	9B 8K Very fast Instruction following

Lec 11: Creating APIs for App using FastAPI

FastAPI is a modern Python web framework for building REST APIs — fast, automatic docs, and native async support. Wrap your LangChain chains in FastAPI endpoints to serve them as a production API.

```

pip install fastapi uvicorn

# main.py
from fastapi import FastAPI
from pydantic import BaseModel
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()
app = FastAPI(title='LLM API', version='1.0')

# --- LangChain chain ---
llm = ChatGroq(model='llama3-8b-8192', temperature=0.7)
chain = ChatPromptTemplate.from_template(
    'Answer concisely: {question}'
) | llm | StrOutputParser()

# --- Request / Response schemas ---
class QuestionRequest(BaseModel):
    question: str

class AnswerResponse(BaseModel):
    answer: str
    model: str = 'llama3-8b-8192'

# --- Endpoints ---
@app.get('/')
def root():
    return {'status': 'ok', 'message': 'LLM API is running'}

@app.post('/ask', response_model=AnswerResponse)
def ask_question(request: QuestionRequest):
    answer = chain.invoke({'question': request.question})
    return AnswerResponse(answer=answer)

@app.get('/health')
def health(): return {'status': 'healthy'}

# Run: uvicorn main:app --reload
# Docs: http://localhost:8000/docs (Swagger UI auto-generated)

```

Lec 12: Serving Groq App using FastAPI

Production-ready FastAPI setup with streaming support, async endpoints, CORS for frontend access, and LangServe as an alternative deployment method.

Streaming Endpoint

```

from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from pydantic import BaseModel

app = FastAPI()
llm = ChatGroq(model='llama3-8b-8192')
chain = ChatPromptTemplate.from_template('Answer: {q}') | llm | StrOutputParser()

class Q(BaseModel):
    q: str

@app.post('/stream')
async def stream_answer(request: Q):

```

```

    async def generate():
        async for chunk in chain.astream({'q': request.q}):
            yield chunk
    return StreamingResponse(generate(), media_type='text/plain')

# — CORS (allow browser frontend to call API) —————
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(CORSMiddleware,
    allow_origins=['*'], # restrict in production
    allow_methods=['*'],
    allow_headers=['*'])
)

```

LangServe — One-Line Chain Deployment

```

pip install langserve[all]

# server.py
from fastapi import FastAPI
from langserve import add_routes
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

app = FastAPI()
chain = (
    ChatPromptTemplate.from_template('Answer: {question}')
    | ChatGroq(model='llama3-8b-8192')
    | StrOutputParser()
)

add_routes(app, chain, path='/groq')

# Auto-generated endpoints:
# POST /groq/invoke      <- single call
# POST /groq/stream     <- streaming
# POST /groq/batch      <- batch
# GET /groq/playground  <- browser UI with no code

# Run: uvicorn server:app --reload --port 8000

```

Deployment Option	When to Use Pros
FastAPI manual	Full control needed; custom auth, middleware, multi-endpoint Flexible, production-grade
LangServe	Rapid prototyping; expose one chain quickly Zero boilerplate, playground UI
Streamlit	Internal demo / non-technical users No frontend code needed
Docker + FastAPI	Production, scalable deployment Containerised, portable, cloud-deployable

End of Module 30 Notes — Generative AI Course